

Universidade do Minho
Escola de Engenharia

Programação Orientada aos Objetos

Trabalho Prático – DomusControl

Grupo 46

Gabriel Matos Rodrigues a110987 

Luís Araújo Dias a111474 

Luís Pedro Matos Machado a110729 

1. Introdução

No âmbito na disciplina de Programação Orientada aos Objetos foi desenvolvida uma plataforma intitulada ***DomusControl***, focada na simulação e gestão de um ambiente de *Smart Home.*

Neste contexto, o objetivo principal deste projeto consistiu em criar um sistema robusto e escalável capaz de administrar múltiplas casas, as suas respetivas divisões e uma vasta gama de dispositivos inteligentes, desde Lâmpadas, Colunas de som, Reles, Cortinas, Portões de Garagem e Tomadas.

Para além da gestão física do espaço, o sistema foi desenhado para suportar a interação de diferentes perfis de utilizadores (com privilégios de Administrador ou Utilizador comum) e permitir a criação e implementação de automações, de cenários e escalonamentos, conferindo autonomia e inteligência à habitação.

Foi desenvolvida um menu interface para permitir o aos utilizadores operar sobre os dispositivos existentes ou criar novos, e ainda definir automações, escalonamentos e cenários. É possível ainda obter informações sobre o sistema através do uso dos métodos estatísticos acessíveis pelo menu.

2. Arquitetura do Sistema

A conceção da arquitetura do ***DomusManager*** foi pautada pelos princípios fundamentais do paradigma de Programação Orientada aos Objetos, visando maximizar a coesão interna das classes e minimizar o acoplamento entre os diferentes módulos. A topologia do domínio foi modelada de forma a garantir uma separação de responsabilidades, isolando a lógica da camada de apresentação. Desta forma, foi desenvolvido um diagrama de classes UML usando um software próprio para esta finalidade, chamado *StarUML*.

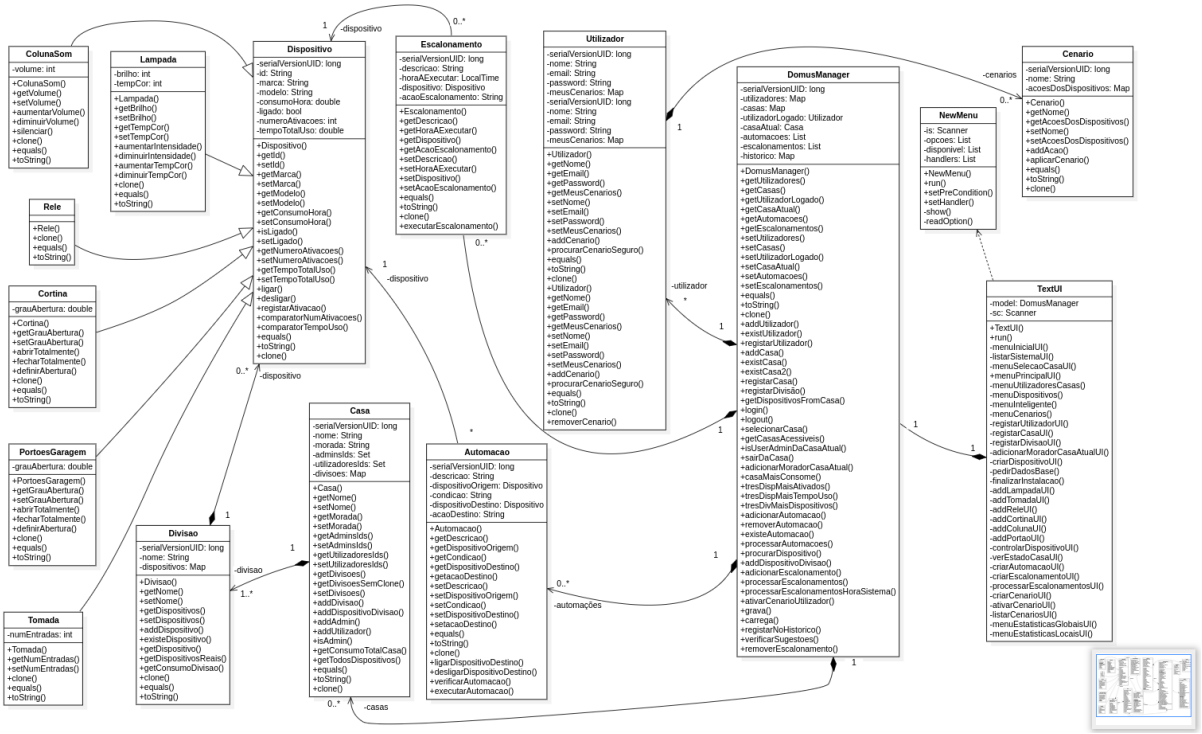


Figura 1 – Diagrama de classes UML do *DomusControl*

Esta figura encontra-se indexada no final deste documento para melhor visualização.

2.1. DomusControl Manager

O núcleo operacional do sistema realiza-se na classe *DomusManager*. Esta classe desempenha a função principal do sistema, sendo responsável por guardar e gerir todas as informações principais, como os utilizadores registados, as casas criadas, as automações e os escalonamentos, as divisões das casas e os dispositivos que estão associados às mesmas.

A relação entre o *DomusManager* e estas entidades é uma Composição, o que significa que a existência destas depende da existência do gestor central, pelo que, se o gestor for encerrado ou apagado, a informação que este coordena também deixa de existir na memória.

2.2. Estrutura Física

A organização física do sistema está dividida em duas partes a casa e as divisões da casa, o que reflete a realidade de uma habitação comum.

Casa: atua como o edifício principal. Cumprindo o requisito de controlo de acessos, a casa mantém um registo de quais são os utilizadores que têm permissão de Administrador ou de Utilizador comum.

Divisão: representa os compartimentos. Como o enunciado obrigava a que os aparelhos estivessem organizados por salas, a Divisão foi ligada à Casa através de uma relação de Composição. Isto garante a regra lógica de que uma divisão não pode existir no sistema sem pertencer a uma casa, e é dentro desta divisão que os aparelhos ficam fisicamente instalados.

2.3. Dispositivos e Herança

Um dos requisitos mais fortes do enunciado era a existência de vários tipos de aparelhos inteligentes (lâmpadas, tomadas, portões, etc.), que deviam ter todos umas características comuns (por exemplo, uma variável binária de ligado ou desligado), mas também tivessem características únicas (como a cor de uma lâmpada ou o volume de uma coluna).

Para cumprir este ponto sem repetir código, utilizámos o conceito de Herança. Criámos uma classe pai, Dispositivo, que agrupa todas as características obrigatórias definidas no enunciado para qualquer aparelho (identificador, consumo energético e estado). A partir desta classe, derivámos as classes filhas específicas (Lampada, Tomada, ColunaSom, etc.), que acrescentam apenas os seus atributos próprios.

Esta decisão arquitetural permitiu-nos cumprir o requisito de ligar/desligar vários equipamentos diferentes de uma só vez, tirando partido do Polimorfismo. O sistema consegue agrupar lâmpadas e cortinas na mesma lista da divisão e enviar o comando ligar para todos, sem precisar de saber que tipo de aparelho específico está a manipular.

2.4. A Interface com o Utilizador

Por fim, o enunciado solicitava uma interface interativa em modo de texto e valorizava as boas práticas de programação, como a separação entre a interface gráfica e o motor do programa.

Desta forma, isolámos toda a interação com o utilizador na classe TextUI. O gestor do programa, *DomusManager*, nunca faz perguntas diretamente ao utilizador, é o TextUI que desenha os menus, lê as opções escolhidas e, de seguida, envia as ordens corretas para o gestor central. Para organizar os vários ecrãs exigidos, o TextUI compõe-se de vários objetos da classe Menu, garantindo uma navegação fluida e sem erros.

3. Descrição da Aplicação e Funcionalidades

O sistema ***DomusControl*** foi desenvolvido com uma Interface de Linha de Comandos, baseada em menus interativos, garantindo uma navegação intuitiva e fluida para o utilizador. As principais funcionalidades do sistema dividem-se em quatro grandes eixos:

3.1. Gestão de Entidades e Infraestrutura Base

O alicerce da aplicação foca-se na estruturação lógica da casa e nos seus moradores:

- O sistema suporta múltiplos perfis, garantindo a autenticação através de email e password. Os utilizadores podem pertencer a várias casas, existindo uma distinção clara na partilha de funcionalidades, dependendo do tipo de utilizador (Admin ou Utilizador Normal).
- É possível criar e gerir múltiplas casas. Cada casa é subdividida em divisões (ex: Sala, Quarto, Garagem).
- O utilizador pode instalar aparelhos inteligentes nas divisões, tais como:
  - *Lâmpadas* (com controlo de brilho e temperatura de cor),
  - *Cortinas (com controlo do grau de abertura);*
  - *Colunas de Som (com controlo do volume);*
  - *Portões de Garagem (com controlo do grau de abertura);*
  - *Tomadas* e *Relés.*

A arquitetura permite ligar, desligar e ajustar os parâmetros específicos de cada equipamento.

3.2. Lógica de Controlo e Inteligência do Sistema

Para dar resposta às exigências do enunciado e tornar a casa verdadeiramente "inteligente", focamo-nos em desenvolver um sistema que não precisasse de intervenção manual constante. Para não estarmos a repetir código, implementámos primeiro um mecanismo de execução universal na nossa lógica.

Em vez de limitarmos as ações apenas a um verdadeiro/falso (ligar ou desligar), o nosso sistema lê as ações em formato de String (por exemplo, "abertura:50", "brilho:100" ou "ligar"). O sistema faz o *split* (divisão) desta string e, utilizando os conceitos de Herança e Polimorfismo (juntamente com o *instanceof*), verifica em tempo real qual é a classe do dispositivo que vai receber a ação e aplica o método correto (ex: se for um PortoesGaragem, aplica o definirAbertura()).

Esta funcionalidade base é partilhada por três grandes funcionalidades da nossa aplicação:

3.2.1. Cenários

Implementámos os cenários para permitir que várias ações ocorram ao mesmo tempo, tendo em conta o cenário selecionado/criado.

Um cenário guarda uma lista de dispositivos e as respetivas ações que devem sofrer. Os cenários são guardados ao nível do Utilizador (e não da Casa) utilizando um HashMap, garantindo o encapsulamento, uma vez que as rotinas de um utilizador (ex: o cenário de "Deitar", “Acordar” ou "Jantar com amigos") são pessoais. Esta decisão de arquitetura permite uma procura eficiente através do nome do cenário.

Ainda para respeitar rigorosamente o princípio do encapsulamento, implementámos o método clone() na classe Cenario. Sempre que um cenário é guardado ou recuperado do Map, o sistema cria uma cópia profunda dos dados. Isto serve de segurança evitando que alterações acidentais em objetos na memória corrompam as definições originais do cenário guardado.

Quando um cenário é ativado, o sistema percorre a sua lista interna de ações e ordena a execução das ações. Isto permite que, com um único comando, o utilizador altere o estado de dispositivos de classes diferentes (ex: desligar uma Lampada e fechar um PortoesGaragem) de forma atómica e segura.

O utilizador pode também apagar os cenários, sendo que nós garantimos que a remoção é feita de forma segura no *Map*.

```
 ************************************
 *** MENU DA CASA: CASA INTELIGENTE ***
 ************************************
1 - Gestão de Casas e Utilizadores
2 - Gestão de Dispositivos
3 - Automações e Escalonamentos
4 - Cenários
5 - Ver Estado da Casa
6 - Ver Estatísticas da Casa
7 - Gravar Estado Atual
8 - Trocar de Casa (Voltar atrás)
0 - Sair do Programa
Opção: 4

 ********************
 *** DomusControl ***
 ********************

1 - Criar Novo Cenário
2 - Ativar Cenário Existente
3 - Listar Meus Cenários
4 - Remover Cenários
0 - Voltar atrás
Opção: 3

--- Meus Cenários Disponíveis ---
Cenário: 'Acordar'
    └ Dispositivo [004] -> Ação: abertura: 100
    └ Dispositivo [008] -> Ação: abertura: 100
    └ Dispositivo [005] -> Ação: ligar  |   volume: 30

Cenário: 'Deitar'
    └ Dispositivo [008] -> Ação: abertura: 0
    └ Dispositivo [001] -> Ação: desligar
    └ Dispositivo [003] -> Ação: desligar
    └ Dispositivo [006] -> Ação: ligar  |   brilho: 10  |   cor: 2700

Cenário: 'Sair de Casa'
    └ Dispositivo [001] -> Ação: desligar
    └ Dispositivo [002] -> Ação: desligar
    └ Dispositivo [003] -> Ação: desligar
    └ Dispositivo [004] -> Ação: desligar
    └ Dispositivo [005] -> Ação: desligar  |   desligar
    └ Dispositivo [006] -> Ação: desligar
    └ Dispositivo [007] -> Ação: desligar
    └ Dispositivo [008] -> Ação: desligar
    └ Dispositivo [009] -> Ação: desligar

Cenário: 'Jantar com Amigos'
    └ Dispositivo [004] -> Ação: abertura: 50
    └ Dispositivo [005] -> Ação: ligar  |   volume: 50
    └ Dispositivo [003] -> Ação: ligar  |   brilho: 75  |   cor: 3000
```

Figura 2 - Visualização do menu de gestão de Cenários

3.2.2. Escalonamentos

Os escalonamentos foram desenvolvidos para agendar ações a efetuar com base a hora do dia.

O utilizador define uma hora (utilizando a classe *LocalTime)*, um dispositivo de destino e a ação desejada. Cada objeto Escalonamento guarda cada um destes campos anteriormente referidos.

Na nossa interface, criámos a opção de "Processar Escalonamentos", que permite simular uma hora específica do sistema. Quando o método de processamento corre, este compara a hora simulada com a hora agendada. Se coincidirem, o escalonamento é então processado e executa a alteração no dispositivo.

Os escalonamentos são objetos persistentes que ficam registados na lista de tarefas da casa. Isto garante que, mesmo após reiniciar o sistema e carregar o estado, as rotinas temporais configuradas permaneçam ativas e prontas a serem disparadas quando a condição horária for atingida.

```
 ************************************
 *** MENU DA CASA: CASA INTELIGENTE ***
 ************************************
1 - Gestão de Casas e Utilizadores
2 - Gestão de Dispositivos
3 - Automações e Escalonamentos
4 - Cenários
5 - Ver Estado da Casa
6 - Ver Estatísticas da Casa
7 - Gravar Estado Atual
8 - Trocar de Casa (Voltar atrás)
0 - Sair do Programa
Opção: 3

 ********************
 *** DomusControl ***
 ********************

1 - Criar Automação
2 - Listar Automações
3 - Processar Automações
4 - Eliminar Automações
--------------------------
5 - Criar Escalonamento
6 - Listar Escalonamentos
7 - Processar Escalonamentos
8 - Eliminar Escalonamentos
--------------------------
9 - Ver Sugestões Inteligentes
0 - Voltar atrás
Opção: 6

==============================
 MEUS ESCALONAMENTOS AGENDADOS
==============================

 09:00 | Abrir Portao Garagem -> PortoesGaragem [002] | Ação: 1
 09:00 | Abrir PortaoGaragem de manha -> PortoesGaragem [002] | Ação: abertura: 100
 10:00 | Desligar coluna para reunião -> ColunaSom [005] | Ação: desligar
 19:00 | Ligar lampada garagem ao fim do dia -> Lampada [006] | Ação: ligar
```

Figura 3 - Visualização do menu de gestão de Escalonamentos

3.2.2.1 Sugestões Inteligentes

Implementámos ainda o módulo de Sugestões Inteligentes, desenhado para aprender com os hábitos manuais do utilizador e automatizar a sua rotina diária.

O sistema atua como um observador das interações do utilizador. Cada vez que este opera um dispositivo de forma manual através do menu (ex: Desligar uma Lâmpada), o sistema regista essa ação e o bloco horário em que ocorreu (ex: 14h). Se o sistema detetar que o utilizador repetiu exatamente a mesma ação, no mesmo equipamento e na mesma faixa horária por três vezes (simulando um hábito diário), este comportamento é sinalizado como um padrão recorrente.

Para concretizar esta funcionalidade de forma altamente eficiente em memória, criamos a funcão *registarNoHistorico,* na classe DomusManager e utilizámos um *HashMap<String, Integer>*. A chave do *Map* é uma *String* composta que concatena três variáveis: o ID do dispositivo, a ação executada e a hora exata (id:acao:hora, resultando em chaves como "001:Desligar:14"). O valor associado é o contador de ocorrências (*Integer)*, que é incrementado a cada interação correspondente.

Ao aceder ao menu de Sugestões Inteligentes, o sistema varre o histórico em busca de contadores com valor igual ou superior a 3. Quando encontra uma correspondência, informa o utilizador do padrão detetado (ex: *"costuma Desligar o dispositivo [001] por volta das 14:00"*) e questiona se este deseja automatizar a tarefa. Se a resposta for afirmativa, o sistema reaproveita a arquitetura de código existente e gera imediatamente um novo objeto Escalonamento, reiniciando o contador daquela chave para evitar sugestões duplicadas no futuro.

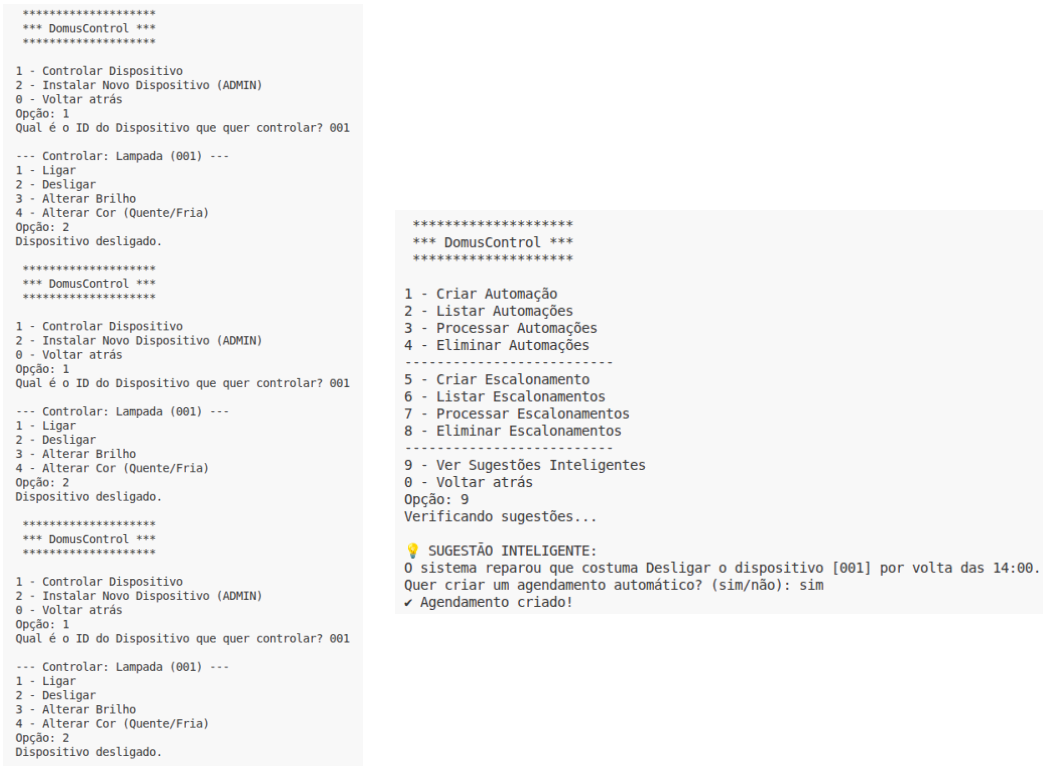


Figura 4 - Visualização do Funcionamento das Sugestões Inteligentes

3.2.3. Automações

Implementamos as automações com o objetivo de que a mudança de estado de um equipamento provocasse uma reação automática noutro, sem o utilizador ter de intervir.

Em vez de criarmos classes de dispositivos exclusivamente dedicadas a serem "Sensores", tomámos a decisão de a arquitetura permitir que qualquer dispositivo pudesse atuar como um gatilho. Isto é possível graças ao conceito de Herança: como todos os equipamentos (Lâmpadas, Cortinas, Portões, Relés, ...) herdam da superclasse Dispositivo, todos partilham um estado base binário na memória (Ligado/Desligado).

Na nossa classe *Automacao,* não guardamos apenas os Ids escritos em texto, mas sim as referências aos objetos reais do dispositivo de Origem e do dispositivo de Destino. Guardamos também a condição esperada (uma String, ex: "ligado") e a ação a aplicar no destino.

Na nossa interface, no menu das Automações, quando o utilizador escolhe a opção "Processar Automações", o sistema percorre a lista de regras ativas na Casa. Para cada regra, o método *verificarAutomacao()* vai invocar o *isLigado()* do objeto de Origem. Se o estado real do aparelho coincidir com a condição guardada (por exemplo, a origem está efetivamente "ligada"), a automação é então validada e processada.

Por exemplo, o utilizador pode criar uma automação onde a Origem é o PortoesGaragem com a condição "ligado" (que fisicamente se traduz na abertura do portão), e o Destino é a Lampada da garagem com a ação "brilho:100". Quando a automação é ativada, ela passa a instrução a executar ao "mecanismo universal" que implementamos. Este faz o *split* da string, usa o *instanceof* para confirmar que o destino é de facto uma Lampada (Polimorfismo), e invoca o método de alterar o brilho.

```
 ************************************
 *** MENU DA CASA: CASA INTELIGENTE ***
 ************************************
1 - Gestão de Casas e Utilizadores
2 - Gestão de Dispositivos
3 - Automações e Escalonamentos
4 - Cenários
5 - Ver Estado da Casa
6 - Ver Estatísticas da Casa
7 - Gravar Estado Atual
8 - Trocar de Casa (Voltar atrás)
0 - Sair do Programa
Opção: 3

 ********************
 *** DomusControl ***
 ********************

1 - Criar Automação
2 - Listar Automações
3 - Processar Automações
4 - Eliminar Automações
-------------------------
5 - Criar Escalonamento
6 - Listar Escalonamentos
7 - Processar Escalonamentos
8 - Eliminar Escalonamentos
-------------------------
9 - Ver Sugestões Inteligentes
0 - Voltar atrás
Opção: 2

=========================
 MINHAS AUTOMAÇÕES ATIVAS
=========================

 Reduzir luz ao fechar as cortinas  | Se Cortina [004] desligado -> Atuar em Lampada [003] com Ação: brilho: 30
 Ligar aquecedor quando se fecham as cortinas | Se Cortina [004] desligado -> Atuar em Rele [009] com Ação: ligar
```

Figura 5 - Visualização do menu de gestão de Automações

3.3. Monitorização e Estatísticas

Quando o utilizador solicita a visualização do estado da casa, o sistema executa uma travessia hierárquica pelas coleções de dados. O objeto Casa percorre o seu Map de Divisões, e cada Divisao percorre a sua lista de Dispositivos. O aproveitamento da Herança é, mais uma vez, crucial aqui, dado que são invocados os métodos polimórficos (como o toString() ou métodos de leitura de estado) de cada objeto filho. Assim, a interface gráfica consegue apresentar dados muito específicos (como a temperatura de cor de uma Lâmpada ou a percentagem de abertura de uma Cortina) sem necessidade de código repetitivo, resultando numa formatação limpa e organizada por divisões.

```
 ************************************
 *** MENU DA CASA: CASA INTELIGENTE ***
 ************************************
1 - Gestão de Casas e Utilizadores
2 - Gestão de Dispositivos
3 - Automações e Escalonamentos
4 - Cenários
5 - Ver Estado da Casa
6 - Ver Estatísticas da Casa
7 - Gravar Estado Atual
8 - Trocar de Casa (Voltar atrás)
0 - Sair do Programa
Opção: 5

================================================
 CASA: CASA INTELIGENTE | MORADA: Rua 24 de Maio
================================================

 ----------------------------------
 Divisão: GARAGEM (2 dispositivos)
 ----------------------------------
  [Lampada]          ID: 001    | Estado: ON  | Consumo: 10Wh | Brilho: 100 | TempCor: 3500K
  [PortoesGaragem]   ID: 002    | Estado: ON  | Consumo: 100Wh | Grau de Abertura: 0%

 --------------------------------
 Divisão: SALA (4 dispositivos)
 --------------------------------
  [Lampada]          ID: 003    | Estado: ON  | Consumo: 9Wh | Brilho: 99 | TempCor: 4000K
  [Cortina]          ID: 004    | Estado: ON  | Consumo: 30Wh | Abertura: 40%
  [ColunaSom]        ID: 005    | Estado: ON  | Consumo: 50Wh | Volume: 0%
  [Rele]             ID: 009    | Estado: OFF | Consumo: 30Wh

 ----------------------------------
 Divisão: QUARTO (3 dispositivos)
 ----------------------------------
  [Lampada]          ID: 006    | Estado: ON  | Consumo: 9Wh | Brilho: 100 | TempCor: 3800K
  [Tomada]           ID: 007    | Estado: ON  | Consumo: 25Wh | Número de Entradas: 2
  [Cortina]          ID: 008    | Estado: ON  | Consumo: 39Wh | Abertura: 100%

================================
 CONSUMO TOTAL ATUAL: 0.272 kWh
================================


==============================
Pressione ENTER para voltar...
```

Figura 6 – Menu com funcionalidade de listagem ativa

Quanto às estatísticas, para garantir uma análise abrangente, as métricas estatísticas foram divididas em duas vertentes: Globais (nível do sistema geral) e específicas (nível da habitação).

1) Estatísticas Globais do Sistema: Estas estatísticas agregam informações transversais a todos os utilizadores e casas geridas pelo *DomusManager*:

- Casa com Maior Consumo Energético: O sistema itera sobre o Map de todas as casas registadas, interrogando cada uma sobre o seu consumo total acumulado no momento. O algoritmo identifica o valor máximo e converte a métrica resultante para quilowatts-hora (kWh), fornecendo assim uma leitura realista.

- Top 3 Divisões com Mais Dispositivos: O algoritmo executa uma travessia completa à árvore de dados (Sistema - Casas - Divisões). Ao aceder às listas de equipamentos de cada divisão, o sistema extrai o seu tamanho (*size()*) e ordena as instâncias. Isto permite comparar divisões de habitações completamente distintas num único ranking.

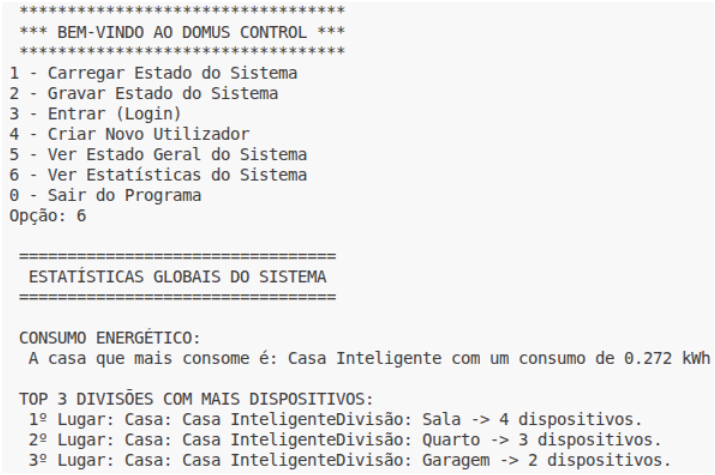


Figura 7 – Menu das Estatísticas Globais do Sistema

2) Estatísticas Específicas da Casa: A visualização específica permite a cada utilizador focar-se no comportamento detalhado da casa.

- Memória de Estado e Encapsulamento: Para suportar métricas precisas ao nível do hardware, a arquitetura exigiu a inclusão de memória de estado. A superclasse Dispositivo foi expandida para incluir atributos que registam o histórico de uso, nomeadamente um contador de interações (vezes que o dispositivo foi ligado) e um acumulador do tempo de utilização total.

- Algoritmos de Ordenação (Top 3 Dispositivos): Em vez de iterar manualmente por várias listas, o sistema utiliza mecanismos avançados de ordenação e filtragem do Java (recorrendo à interface Comparator e ordenação de coleções). Assim,

ao extrair todos os aparelhos de todas as divisões de uma casa, o motor consegue ordenar e devolver com alta eficiência o **Top 3 de Dispositivos Mais Ativados** e o **Top 3 de Dispositivos com Mais Tempo de Uso**. Esta abordagem computacional garante a gestão otimizada da memória, evitando o armazenamento de listas intermédias pesadas.

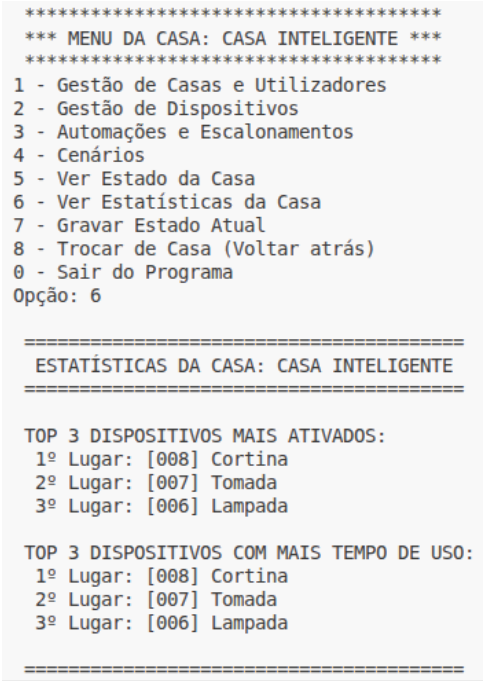


Figura 8 – Menu das Estatísticas Específicas de cada Casa

3.4. Persistência de Dados (Estado do Sistema)

Para garantir que nenhuma configuração é perdida entre sessões, a aplicação implementa a serialização de objetos.

Cada classe implementa então o *serializable* da linguagem Java, que indica que um objeto pode ser convertido numa sequência de bytes, possibilitando o seu armazenamento em ficheiros e posterior recuperação. No caso dos dispositivos, apenas a classe Dispositivo implementa esta interface, uma vez que todas as restantes classes de dispositivos herdam dela e, consequentemente, também herdam a capacidade de serialização.

Adicionalmente foi ainda adicionado a todas estas classes o atributo *private static final long serialVersionUID = 1L;*

Este identificador é utilizado pelo Java para controlar a compatibilidade entre diferentes versões serializadas da mesma classe. Assim, garante-se que objetos guardados anteriormente podem ser lidos corretamente, desde que a estrutura da classe permaneça compatível.

```
public class DomusManager implements Serializable {
public abstract class Dispositivo implements Serializable {
    private static final long serialVersionUID = 1L;
```

Figura 9 – Exemplo de implementação do *serializable*

O utilizador pode gravar o Estado Atual do sistema num ficheiro binário local (ex: estado.dat). Ao reiniciar a aplicação, é possível Carregar o Estado, recuperando instantaneamente todo o sistema de utilizadores, casas, dispositivos, e todas as lógicas de automações, escalonamentos e cenários previamente configuradas.

Na interface desenvolvida a implementação da funcionalidade de carregar/guardar um estado, foi projetada de forma a melhorar a experiência do utilizador. Isto porque, logo ao iniciar a aplicação, o utilizador tem a possibilidade de "Carregar o Estado do Sistema", o que permite retomar instantaneamente todo o ecossistema de casas e utilizadores. Existe também a opção de "Gravar" neste nível, útil para salvaguardar os dados logo após o login ou após a criação de novos utilizadores no sistema.

Dentro do menu específico de uma casa, foi incluída a opção "Gravar Estado Atual". Esta decisão de design permite que, após o utilizador configurar uma automação, um cenário, um escalonamento, um novo dispositivo, entre outros... possa persistir essas alterações imediatamente sem ter de retroceder nos menus.

Esta estrutura garante que o "estado da memória" do sistema possa ser capturado em qualquer momento crítico da interação, prevenindo a perda de dados em caso de encerramento inesperado e facilitando a gestão contínua de habitações inteligentes a longo prazo.

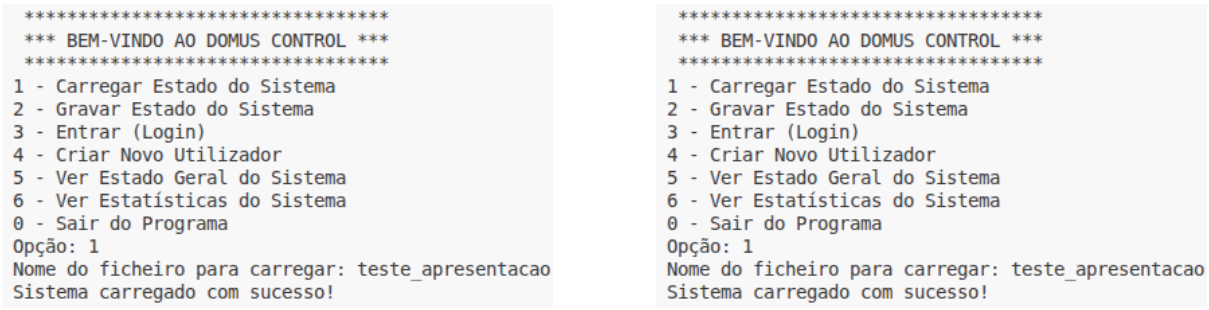


Figura 10 – Guardar/Carregar no Menu Inicial

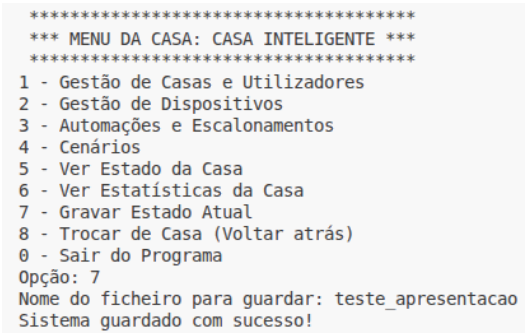


Figura 11 – Guardar no Menu da Casa

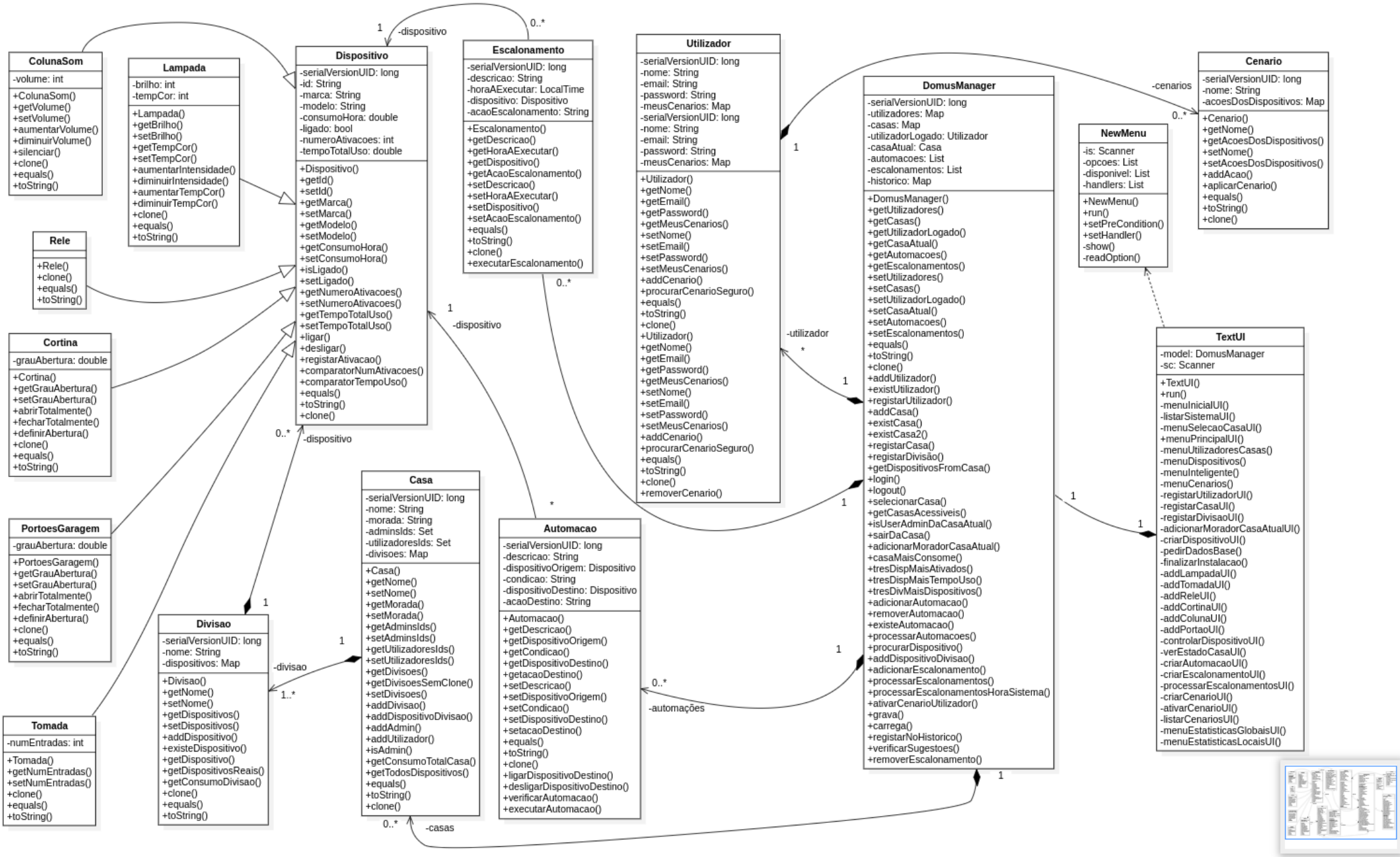
4. Conclusão

O desenvolvimento do sistema **DomusControl** revelou-se um excelente desafio prático para a consolidação dos conceitos de Programação Orientada aos Objetos. O projeto cumpriu todos os requisitos estipulados no enunciado, resultando numa aplicação de consola robusta, interativa e capaz de simular a complexidade de uma habitação inteligente.

A utilização intensiva de **Herança** e **Polimorfismo** provou ser a decisão arquitetural mais acertada. Ao basear todo o sistema numa superclasse genérica de Dispositivo, conseguimos eliminar a repetição de código e criar um "Motor Universal" capaz de processar Automações, Escalonamentos e Cenários de forma centralizada e eficiente. Garantimos também o encapsulamento rigoroso dos dados através do uso de cópias profundas e estruturas de dados dinâmicas como o *HashMap.*

O maior desafio técnico centrou-se na lógica reativa das Automações. Garantir que as alterações de estado de um aparelho (Origem) criassem ações precisas noutro (Destino), sem gerar conflitos de memória e lidando com interpretação de strings, exigiu algum cuidado nas referências entre objetos.

Em suma, consideramos que a arquitetura desenvolvida responde de forma exímia ao problema proposto, criando uma base de código modular e extensível.


ColunaSom
-volume: int
+ColunaSom()
+getVolume()
+setVolume()
+aumentarVolume()
+diminuirVolume()
+silenciar()
+clone()
+equals()
+toString()
Lampada
-brilho: int
-tempCor: int
+Lampada()
+getBrilho()
+setBrilho()
+getTempCor()
+setTempCor()
+aumentarIntensidade()
+diminuirIntensidade()
+aumentarTempCor()
+diminuirTempCor()
+clone()
+equals()
+toString()
Rele
+Rele()
+clone()
+equals()
+toString()
Cortina
-grauAbertura: double
+Cortina()
+getGrauAbertura()
+setGrauAbertura()
+abrirTotalmente()
+fecharTotalmente()
+definirAbertura()
+clone()
+equals()
+toString()
PortoesGaragem
-grauAbertura: double
+PortoesGaragem()
+getGrauAbertura()
+setGrauAbertura()
+abrirTotalmente()
+fecharTotalmente()
+definirAbertura()
+clone()
+equals()
+toString()
Tomada
-numEntradas: int
+Tomada()
+getNumEntradas()
+setNumEntradas()
+clone()
+equals()
+toString()
Dispositivo
-serialVersionUID: long
-id: String
-marca: String
-modelo: String
-consumoHora: double
-ligado: bool
-numeroAtivacoes: int
-tempoTotalUso: double
+Dispositivo()
+getId()
+setId()
+getMarca()
+setMarca()
+getModelo()
+setModelo()
+getConsumoHora()
+setConsumoHora()
+isLigado()
+setLigado()
+getNumeroAtivacoes()
+setNumeroAtivacoes()
+getTempoTotalUso()
+setTempoTotalUso()
+ligar()
+desligar()
+registarAtivacao()
+comparatorNumAtivacoes()
+comparatorTempoUso()
+equals()
+toString()
+clone()
Escalonamento
-serialVersionUID: long
-descricao: String
-horaAExecutar: LocalTime
-dispositivo: Dispositivo
-acaoEscalonamento: String
+Escalonamento()
+getDescricao()
+getHoraAExecutar()
+getDispositivo()
+getAcaoEscalonamento()
+setDescricao()
+setHoraAExecutar()
+setDispositivo()
+setAcaoEscalonamento()
+equals()
+toString()
+clone()
+executarEscalonamento()
Utilizador
-serialVersionUID: long
-nome: String
-email: String
-password: String
-meusCenarios: Map
-serialVersionUID: long
-nome: String
-email: String
-password: String
-meusCenarios: Map
+Utilizador()
+getNome()
+getEmail()
+getPassword()
+getMeusCenarios()
+setNome()
+setEmail()
+setPassword()
+setMeusCenarios()
+addCenario()
+procurarCenarioSeguro()
+equals()
+toString()
+clone()
+Utilizador()
+getNome()
+getEmail()
+getPassword()
+getMeusCenarios()
+setNome()
+setEmail()
+setPassword()
+setMeusCenarios()
+addCenario()
+procurarCenarioSeguro()
+equals()
+toString()
+clone()
+removerCenario()
DomusManager
-serialVersionUID: long
-utilizadores: Map
-casas: Map
-utilizadorLogado: Utilizador
-casaAtual: Casa
-automacoes: List
-escalonamentos: List
-historico: Map
+DomusManager()
+getUtilizadores()
+getCasas()
+getUtilizadorLogado()
+getCasaAtual()
+getAutomacoes()
+getEscalonamentos()
+setUtilizadores()
+setCasas()
+setUtilizadorLogado()
+setCasaAtual()
+setAutomacoes()
+setEscalonamentos()
+equals()
+toString()
+clone()
+addUtilizador()
+existUtilizador()
+registarUtilizador()
+addCasa()
+existCasa()
+existCasa2()
+registarCasa()
+registarDivisão()
+getDispositivosFromCasa()
+login()
+logout()
+selecionarCasa()
+getCasasAcessiveis()
+isUserAdminDaCasaAtual()
+sairDaCasa()
+adicionarMoradorCasaAtual()
+casaMaisConsome()
+tresDispMaisAtivados()
+tresDispMaisTempoUso()
+tresDivMaisDispositivos()
+adicionarAutomacao()
+removerAutomacao()
+existeAutomacao()
+processarAutomacoes()
+procurarDispositivo()
+addDispositivoDivisao()
+adicionarEscalonamento()
+processarEscalonamentos()
+processarEscalonamentosHoraSistema()
+ativarCenarioUtilizador()
+grava()
+carrega()
+registarNoHistorico()
+verificarSugestoes()
+removerEscalonamento()
Cenario
-serialVersionUID: long
-nome: String
-acoesDosDispositivos: Map
+Cenario()
+getNome()
+getAcoesDosDispositivos()
+setNome()
+setAcoesDosDispositivos()
+addAcao()
+aplicarCenario()
+equals()
+toString()
+clone()
NewMenu
-is: Scanner
-opcoes: List
-disponivel: List
-handlers: List
+NewMenu()
+run()
+setPreCondition()
+setHandler()
-show()
-readOption()
TextUI
-model: DomusManager
-sc: Scanner
+TextUI()
+run()
-menuInicialUI()
-listarSistemaUI()
-menuSelecaoCasaUI()
+menuPrincipalUI()
-menuUtilizadoresCasas()
-menuDispositivos()
-menuInteligente()
-menuCenarios()
-registarUtilizadorUI()
-registarCasaUI()
-registarDivisaoUI()
-adicionarMoradorCasaAtualUI()
-criarDispositivoUI()
-pedirDadosBase()
-finalizarInstalacao()
-addLampadaUI()
-addTomadaUI()
-addReleUI()
-addCortinaUI()
-addColunaUI()
-addPortaoUI()
-controlarDispositivoUI()
-verEstadoCasaUI()
-criarAutomacaoUI()
-criarEscalonamentoUI()
-processarEscalonamentosUI()
-criarCenarioUI()
-ativarCenarioUI()
-listarCenariosUI()
-menuEstatisticasGlobaisUI()
-menuEstatisticasLocaisUI()
Divisao
-serialVersionUID: long
-nome: String
-dispositivos: Map
+Divisao()
+getNome()
+setNome()
+getDispositivos()
+setDispositivos()
+addDispositivo()
+existeDispositivo()
+getDispositivo()
+getDispositivosReais()
+getConsumoDivisao()
+clone()
+equals()
+toString()
Casa
-serialVersionUID: long
-nome: String
-morada: String
-adminsIds: Set
-utilizadoresIds: Set
-divisoes: Map
+Casa()
+getNome()
+setNome()
+getMorada()
+setMorada()
+getAdminsIds()
+setAdminsIds()
+getUtilizadoresIds()
+setUtilizadoresIds()
+getDivisoes()
+getDivisoesSemClone()
+setDivisoes()
+addDivisao()
+addDispositivoDivisao()
+addAdmin()
+addUtilizador()
+isAdmin()
+getConsumoTotalCasa()
+getTodosDispositivos()
+equals()
+toString()
+clone()
Automacao
-serialVersionUID: long
-descricao: String
-dispositivoOrigem: Dispositivo
-condicao: String
-dispositivoDestino: Dispositivo
-acaoDestino: String
+Automacao()
+getDescricao()
+getDispositivoOrigem()
+getCondicao()
+getDispositivoDestino()
+getacaoDestino()
+setDescricao()
+setDispositivoOrigem()
+setCondicao()
+setDispositivoDestino()
+setacaoDestino()
+equals()
+toString()
+clone()
+ligarDispositivoDestino()
+desligarDispositivoDestino()
+verificarAutomacao()
+executarAutomacao()
1
-dispositivo
0..*
-dispositivo
1
-dispositivo
0..*
-dispositivo
-utilizador
*
1
-cenarios
0..*
1
1
1
1
1
1
-divisao
1..*
1
0..*
-automações
0..*
-casas
1
1
*
*